

Optimiser la base de données dans MYSQL

Optimiser les requêtes SQL

Conseils:

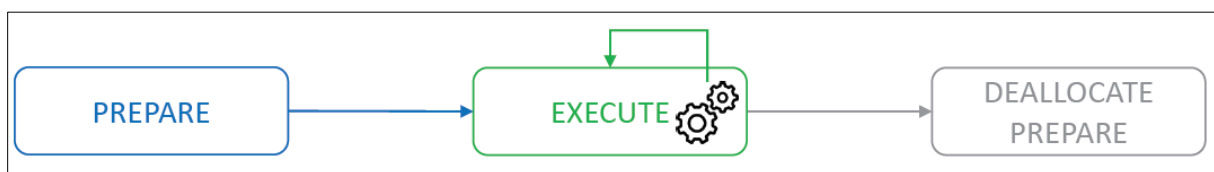
Voici quelques conseils pour optimiser les requêtes SQ :

- Éviter si possible les **SELECT *** et réduire le nombre de champs, afin de réduire les données chargées en mémoire.
- Remplacer les clauses **WHERE ... IN** par **WHERE EXISTS**.
- Éviter d'utiliser des fonctions dans les prédicats : exemple **SELECT * FROM TABLE1 WHERE UPPER(COL1)='ABC'** .
- Éviter d'utiliser un caractère générique (%) au début d'un prédicat **LIKE**.
- Utiliser la jointure interne (**INNER JOIN**), au lieu de la jointure externe (**OUTER JOIN**) si possible.
- N'utiliser **DISTINCT** et **UNION** que si nécessaire.
- N'utiliser **ORDER BY** que si nécessaire.
- Compter les requêtes sur chaque page, un grand nombre de requêtes peut signifier un « problème N+1 », c'est à dire une requête **SELECT** placée dans une boucle.
- Utiliser les requêtes préparées ou les procédures stockées facilite la mise en cache des requêtes en interne par MySQL et assure un bon niveau de sécurité.
- Utiliser la clause **EXPLAIN** pour comprendre le fonctionnement d'une requête et quelles sont les clauses qui impactent ses performances.

Les instructions PREPARE, EXECUTE et DEALLOCATE PREPARE :

L'instruction **PREPARE** permet de préparer une instruction pour être exécuté à l'aide de l'instruction **EXECUTE**.

L'instruction **DEALLOCATE PREPARE** permet de désallouer la préparation. Après cela il n'est plus possible d'exécuter l'instruction préparée.



Voici la syntaxe d'utilisation de la préparation :

```
PREPARE statement_name FROM 'preparable_statement';  
EXECUTE statement_name USING variables_name;  
DEALLOCATE PREPARE statement_name;
```

Exemple 1 :

On va créer la table **employee** suivante :

```
CREATE TABLE employee(id INT PRIMARY KEY, name VARCHAR(255), salary DOUBLE,  
location VARCHAR(255));  
INSERT INTO employee VALUES (101, 'RAJI', 6554, 'Location1');  
INSERT INTO employee VALUES (102, 'ALAMI', 8000, 'Location2');  
INSERT INTO employee VALUES (103, 'BEN ALI', 11265, 'Location1');  
INSERT INTO employee VALUES (104, 'NAJI', 5000, 'Location2');  
INSERT INTO employee VALUES (105, 'RAMI', 4500, 'Location3');  
SELECT * FROM employee;
```

	id	mame	salary	location
▶	101	RAJI	6554	Location1
	102	ALAMI	8000	Location2
	103	BEN ALI	11265	Location1
	104	NAJI	5000	Location2
	105	RAMI	4500	Location3
*	NULL	NULL	NULL	NULL

Maintenant on va tester notre instruction **PREPARE** :

```
PREPARE stm1 FROM 'SELECT * FROM employee';
EXECUTE stm1;
```

	id	mame	salary	location
▶	101	RAJI	6554	Location1
	102	ALAMI	8000	Location2
	103	BEN ALI	11265	Location1
	104	NAJI	5000	Location2
	105	RAMI	4500	Location3
*	NULL	NULL	NULL	NULL

```
DEALLOCATE PREPARE stm1;
EXECUTE stm1;
```

126 17:12:53 EXECUTE stm1 Error Code: 1243. Unknown prepared statement handler (stm1) given to EXECUTE

```
PREPARE stm2 FROM 'SELECT * FROM employee WHERE id=?';
SET @X=102;
EXECUTE stm2 USING @X;
DEALLOCATE PREPARE stm2;
```

	id	mame	salary	location
▶	102	ALAMI	8000	Location2

```
PREPARE stm3 FROM 'INSERT INTO employee VALUES(?,?,?,?)';
SET @id=106, @name='NOURI', @salary=12000, @location='Location1';
EXECUTE stm3 USING @id, @name, @salary, @location;
DEALLOCATE PREPARE stm3;
SELECT * FROM employee;
```

	id	mame	salary	location
▶	101	RAJI	6554	Location1
	102	ALAMI	8000	Location2
	103	BEN ALI	11265	Location1
	104	NAJI	5000	Location2
	105	RAMI	4500	Location3
	106	NOURI	12000	Location1
*	NULL	NULL	NULL	NULL

Les index MYSQL :

Introduction :

Comme l'index d'un ouvrage vous aide à atteindre les pages concernées par un mot recherché, un index MYSQL permet d'accélérer l'accès aux données d'une table.

Lorsqu'une table est créée avec une clé primaire ou une clé unique, elle crée automatiquement un index spécial nommé **PRIMARY** qu'on appelle **clustred index**, et pour les autres index autre que PRIMARY sont appelés **non-clustered index** ou **secondary index**.

Créer un index dans MYSQL :

Voici la syntaxe pour créer un INDEX avec les valeurs en double sont autorisées :

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

Voici la syntaxe pour créer un INDEX avec les valeurs en double ne sont pas autorisées :

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

Exemple :

Avant la définition de l'index :

```
USE w3schools;  
EXPLAIN SELECT * FROM products WHERE ProductName='Konbu';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	products2	NULL	ALL	NULL	NULL	NULL	NULL	79	10.00	Using where

Après la définition de l'index :

```
CREATE INDEX idx_productname ON products2(ProductName);  
EXPLAIN SELECT * FROM products WHERE ProductName='Konbu';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	products2	NULL	ref	product_name	product_name	768	const	1	100.00	NULL

Supprimer un index dans MYSQL :

Deux syntaxes possibles :

```
# Syntaxe 1 :  
DROP INDEX index_name ON table_name;  
# Syntaxe 2 :  
ALTER TABLE table_name  
DROP INDEX index_name;
```

Exemple :

```
DROP INDEX idx_productname ON products;
```